

Free-Pose Road Network Navigation

Free-Pose Road Network Navigation

1. Course Content
2. Running Road Network Planning and Navigation Function
 - 2.1 Start Agent
 - 2.2 Start Sensors and Odometry
 - 2.3 Start Road Network Planning and Navigation Node
 - 2.4 Free-Pose Road Network Planning and Navigation
 - 2.4.1 Application Example
3. Source Code Analysis
 - 3.1 route_bridge.py Analysis

1. Course Content

Based on navigation between waypoints, combine road network planning and navigation functions to achieve navigation functionality that can reach arbitrary poses

[!NOTE]

When navigation2's route_server plans road network navigation, it can only achieve fixed pose navigation. After reaching the target waypoint, it always faces the direction of edges in the road network. In practical applications, we need the robot to face a specified pose when reaching the target point to complete specific tasks. We only use road network planning and navigation as the intermediate global path.

2. Running Road Network Planning and Navigation Function

2.1 Start Agent

- Use the following command to start microROS agent to connect to the chassis

[!WARNING]

The commands for starting agent, sensors and odometry, and road network planning and navigation nodes here are the same as the previous lesson. If already started, **no need to restart**

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
ubsubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

- Modify ROS communication middleware DDS

[!WARNING]

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning and navigation chapter needs to use cyclondds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK & PI5 mainboards

```
131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
145 "
```

- ORIN mainboard

```
133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
```

```
:wq #Save and exit
source ~/.bashrc #Update environment
```

2.2 Start Sensors and Odometry

- Open a terminal on the host to start the vehicle's radar fusion node and odometry fusion
- (For PI5 users, start in the `m3` container)

```
ros2 launch m3_bringup car_base.launch.py
```

```
jetson@yahboom:~$ ros2 launch m3_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2025-11-19-18-43-09-279818-yahboom-1129502
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [launch.user]: =====Robot model ROSMASTER-M3 start=====
[INFO] [launch.user]: =====LASER_TYPE:single=====

[INFO] [launch.user]: LASER start sucefully!,Start by independent node
[INFO] [launch.user]: =====Robot Base Start=====
[INFO] [joint_state_publisher-1]: process started with pid [1129557]
[INFO] [robot_state_publisher-2]: process started with pid [1129559]
[INFO] [lasers_merger-3]: process started with pid [1129561]
[INFO] [scan_to_scan_filter_chain-4]: process started with pid [1129564]
[lasers_merger-3] [INFO] [1763548989.905313662] [laserscan_multi_merger]: Waiting for topics ...
[robot_state_publisher-2] [INFO] [1763548989.924917431] [robot_state_publisher]: got segment ascamera_hp60c_camera
_link_0
[robot_state_publisher-2] [INFO] [1763548989.925142333] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-2] [INFO] [1763548989.925167773] [robot_state_publisher]: got segment base_link
[robot_state_publisher-2] [INFO] [1763548989.925181310] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-2] [INFO] [1763548989.925191422] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-2] [INFO] [1763548989.925200830] [robot_state_publisher]: got segment laser23_frame
[robot_state_publisher-2] [INFO] [1763548989.925209982] [robot_state_publisher]: got segment lwheel1_Link
[robot_state_publisher-2] [INFO] [1763548989.925218911] [robot_state_publisher]: got segment lwheel2_Link
[robot_state_publisher-2] [INFO] [1763548989.925227999] [robot_state_publisher]: got segment rwheel1_Link
[robot_state_publisher-2] [INFO] [1763548989.925236735] [robot_state_publisher]: got segment rwheel2_Link
[scan_to_scan_filter_chain-4] [WARN] [1763548989.935781454] [rcl_logging_rosout]: Publisher already registered for
provided node name. If this is due to multiple nodes with the same name then all logs for that logger name will g
o out over the existing publisher. As soon as any node with that name is destructed it will unregister the publish
er, preventing any further logs for that name from being published on the rosout topic.
[joint_state_publisher-1] [INFO] [1763548990.531349805] [joint_state_publisher]: Waiting for robot_description to
be published on the robot_description topic...
[lasers_merger-3] [INFO] [1763548990.907903719] [laserscan_multi_merger]: Subscribing to topics 2
[scan_to_scan_filter_chain-4] [WARN] [1763548990.924837083] [laser_filter]: diagnostic_updater: No HW_ID was set.
This is probably a bug. Please report it. For devices that do not have a HW_ID, set this value to 'none'. This war
ning only occurs once all diagnostics are OK. It is okay to wait until the device is open before calling setHardwa
reID.
[INFO] [launch.user]: =====Merge_Odom Localization Start=====

[INFO] [ekf_node-5]: process started with pid [1129717]
```

2.3 Start Road Network Planning and Navigation Node

- If the `ros2_kilted` docker container is not started, you need to start the container first

```
bringup_ros2kilted
```

- Open a terminal inside the container

```
exec_ros2kilted
```

- (PI5 & RDK mainboards) Visualization is started on the accompanying virtual machine
`Rosmaster-RoadNet-Ubuntu24.04`,

```
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/nav2_view.rviz
```

- Start road network planning and navigation node

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

[!TIP]

Optional startup parameter description

map The grid map file name to be loaded, default value is `map.yaml`

pose_map The pose map file name to be loaded. When the navigation global positioning method uses slam_toolbox pure localization, the pose map must be passed, default value is:

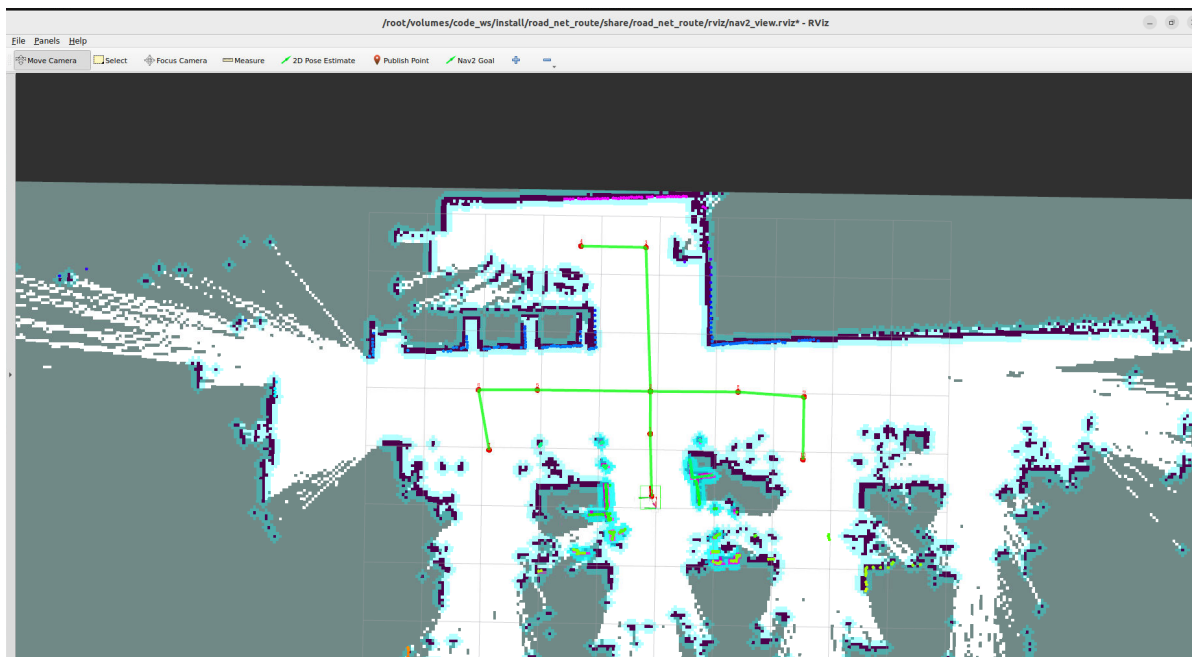
`map`

params_file Navigation parameter file, default value is

`os.path.join(get_package_share_directory('road_net_route'), 'config', 'nav2_kilted.yaml')`

roadnet_file Road network description file, default value is `map/map.geojson`, here the complete file path must be passed

localization_mode Global positioning method, default is `slam_toolbox`



2.4 Free-Pose Road Network Planning and Navigation

- Here we use road network navigation by publishing the `/road_net_nav` topic in the terminal. This topic can accept a `PoseStamped` target pose, travel along the road network to near the target pose, then accurately adjust to the target pose. The topic publishing format is as follows:

```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  pose: {
    position: {x: xx, y: xx, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: xx, w: xx}
  }
}"
```

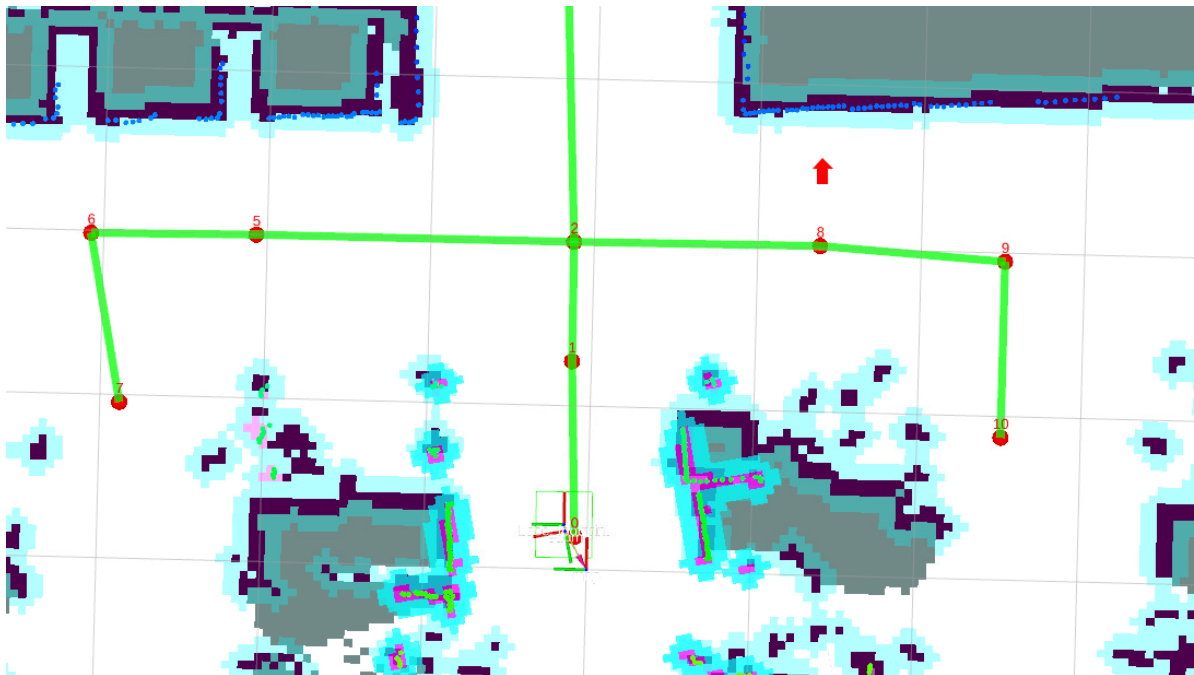
[!NOTE]

Pose consists of position and orientation. For ground mobile robots, position only has planar x,y coordinates, and orientation only rotates around its own z-axis. In pose:

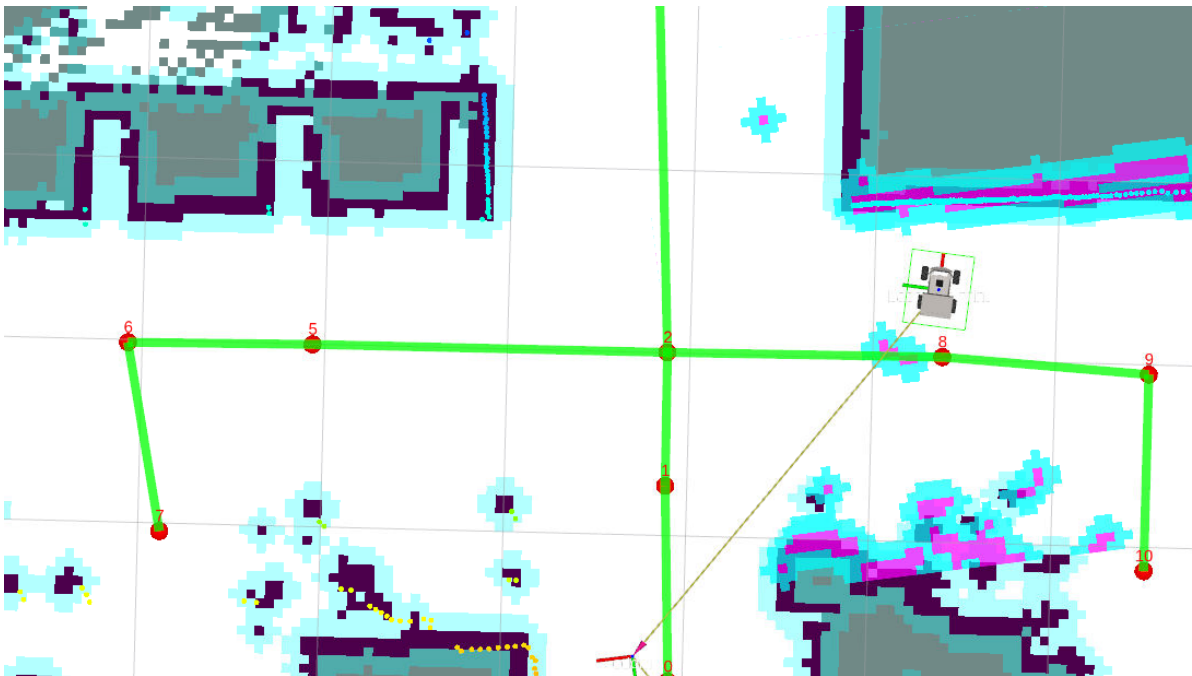
- **position: {x: xx, y: xx, z: 0.0}** is position information
- **orientation: {x: 0.0, y: 0.0, z: xx, w: xx}** is body rotation information described by quaternion

2.4.1 Application Example

- For example, we need the robot vehicle to only travel along the road network, then navigate to a position above waypoint 8 in the map with a specified orientation pose (orientation as shown by the arrow). We need to first know the coordinates of this pose point in the global map.



- First, let the robot move to this target pose



- Use the following command to get the current robot's coordinates in the map

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```

yahboom: ~
jetson@yahboom: ~ 70x19
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.946]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1763608206.239000000
- Translation: [2.383, -1.352, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.052, 0.999]
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.959]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
^C[INFO] [1763608207.095851031] [rclcpp]: signal_handler(signum=2)
jetson@yahboom:~$

```

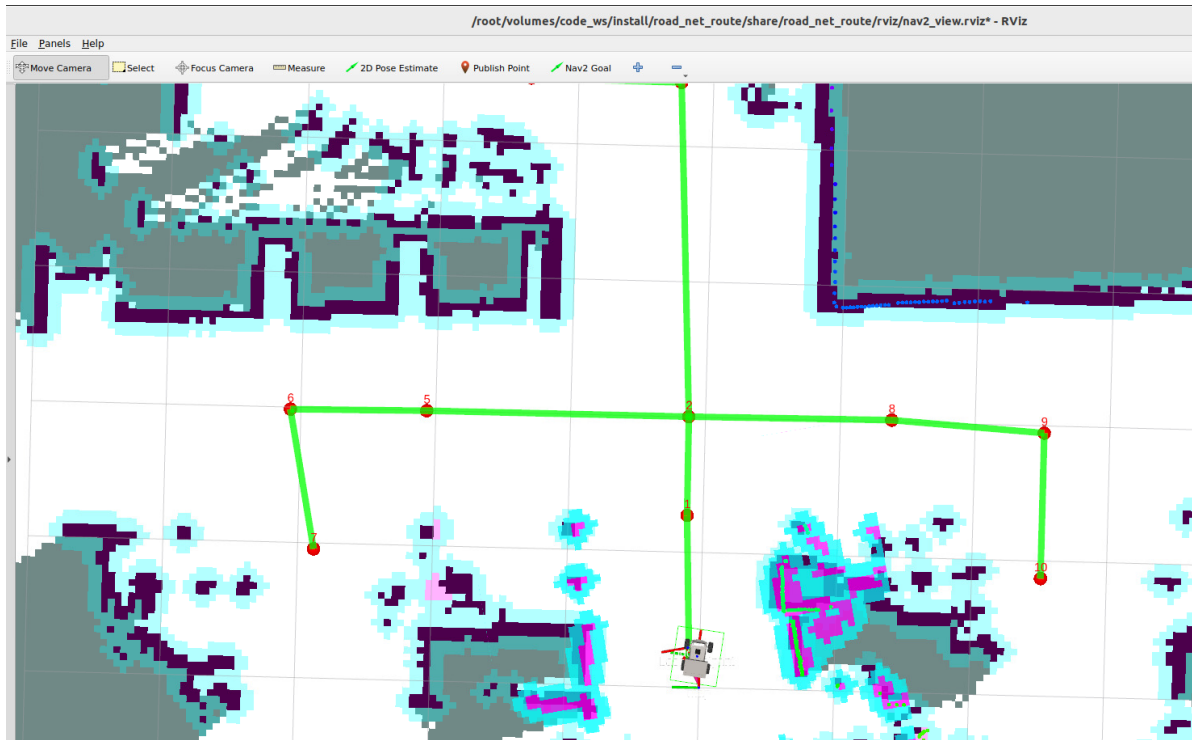
Where `Translation` and `Rotation` are the pose of the robot vehicle in the global map.

- Then let the robot return to the starting position, then publish the target pose in the terminal

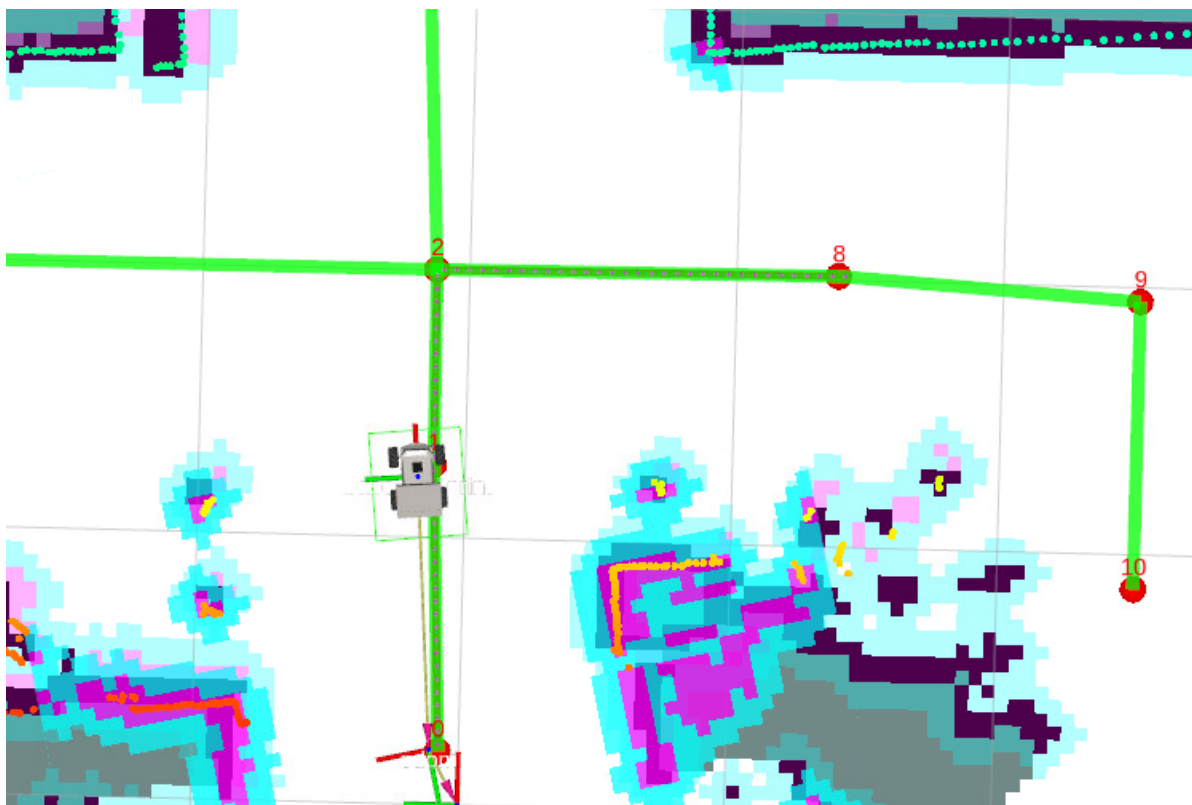
```

ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  header: {frame_id: 'map'},
  pose: {
    position: {x: 2.383, y: -1.352, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: -0.052, w: 0.999}
  }
}"

```

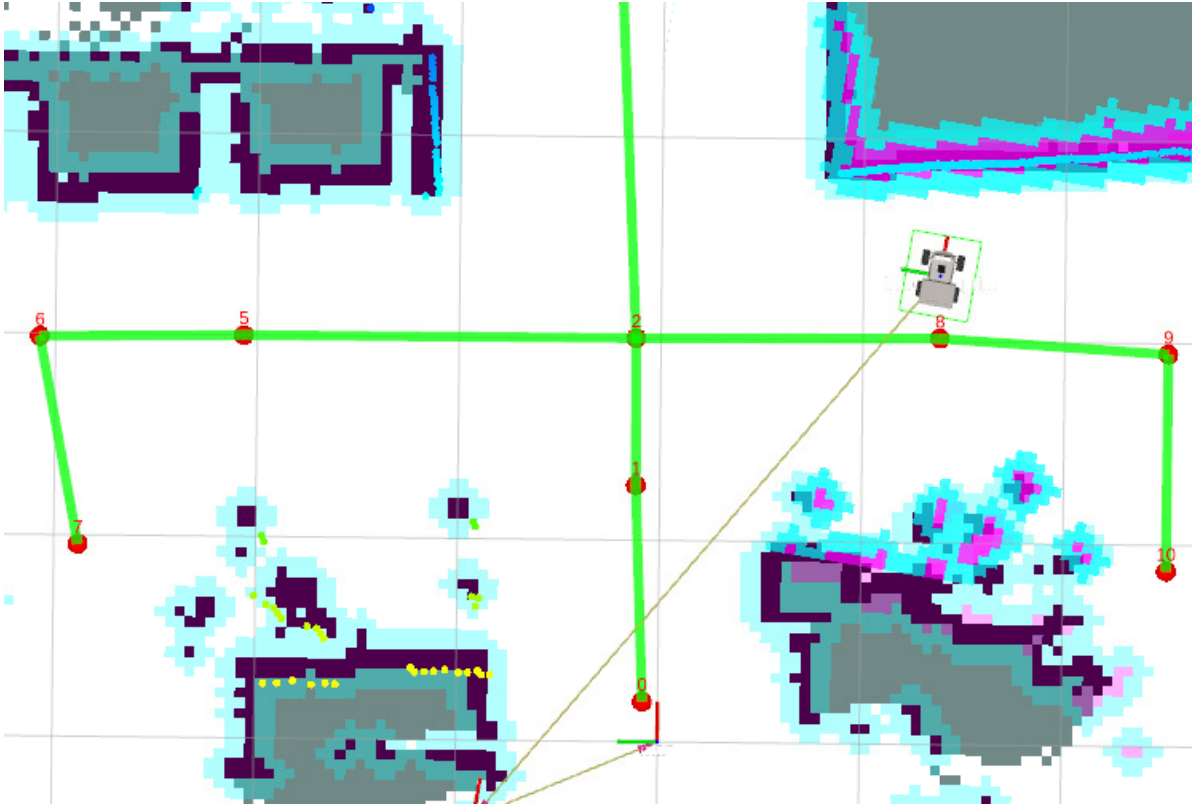


- After that, the robot vehicle will first travel along the road network to a waypoint near the target point



- Finally adjust to the target pose, and the terminal prompts `navigation task completed`

```
[controller_server]: Passing new path to controller.
[component_container_isolated-1] [INFO] [1763609230.200338386] [controller_server]: Reached the goal!
[component_container_isolated-1] [INFO] [1763609230.220298226] [bt_navigator]: Goal succeeded
[route_bridge-4] [INFO] [1763609230.274454384] [route_bridge]: navigation task completed.
```



3. Source Code Analysis

- Implementation program source code path:

```
~/yahboomM3_ws/ros2_kilted/volumes/code_ws/src/road_net_route/road_net_route/route_bridge.py
```

3.1 route_bridge.py Analysis

- In the initialization function of the Route_Bridge class, a `road_net_nav` topic subscriber is created to receive `PoseStamped` data from the `road_net_nav` topic (format is `[start_id, goal_id]`), and directly calls `getAndTrackRoute` to initiate road network navigation:

```
self.road_net_nav_sub= self.create_subscription(PoseStamped, "road_net_nav",
self.nav_callback, 10)
```

- `PoseStamped`: Message type, containing target position and orientation information.
- `"road_net_nav"`: Topic name, through which navigation target pose is sent.
- `self.nav_callback`: Callback function, core logic for processing navigation commands.
- `10`: Message queue size, if callback processing is slow, cache up to 10 messages to avoid message loss.

- Step 1: Get robot current pose (TF transform): Through `tf2_ros`'s `Buffer` and `TransformListener`, get the coordinate transformation from robot base coordinate system (`base_footprint`) to global map coordinate system (`map`), thus obtaining the robot's current pose in the map.
- Step 2: Construct robot current pose: Encapsulate the TF transform result (translation + rotation) as `PoseStamped` type `start_pose`, as the **starting point pose** for navigation.
- Step 3: Pose to road network waypoint ID: This is the **core conversion logic of road network navigation**:
 - Call `__find_nearest_node` method to match **starting point pose** and **target pose** to the nearest node IDs in the road network (through KDTree nearest neighbor search).
 - Call `BasicNavigator`'s `getAndTrackRoute` interface, pass in starting node ID and target node ID, initiate navigation based on road network nodes.
- Step 4: Final pose adjustment: Road network navigation only ensures the robot reaches a waypoint near the target pose. Through `goToPose` interface, let the robot accurately adjust to the target pose.

```
def nav_callback(self, msg: PoseStamped):
    ''' Arbitrary Pose Road Network Navigation Callback Function'''

    transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time(), rclpy.duration.Duration(seconds=5.0))
    if transform is None:
        self.get_logger().info(Fore.RED+"Failed to get TF transform
"+Fore.RESET)
        return
    else:
        start_pose = PoseStamped()
        start_pose.header = transform.header
        start_pose.pose.position.x = transform.transform.translation.x
        start_pose.pose.position.y = transform.transform.translation.y
        start_pose.pose.orientation = transform.transform.rotation
        self.get_logger().info(Fore.CYAN + f"current:
{transform.transform.translation}"+Fore.RESET)

        if self.replace2id:
            nearest_start_node_id, nearest_start_distance =
self.__find_nearest_node(start_pose.pose.position.x, start_pose.pose.position.y)
            nearest_goal_node_id, nearest_goal_distance =
self.__find_nearest_node(msg.pose.position.x, msg.pose.position.y)
            self.get_logger().info(Fore.CYAN + f"nearest_start_node_id:
{nearest_start_node_id}, distance:{nearest_start_distance}"
f"\nnearest_goal_node_id:
{nearest_goal_node_id}, distance:{nearest_goal_distance}"
+Fore.RESET)

            route_tracking_task =
self.navigator.getAndTrackRoute(start=nearest_start_node_id,goal=nearest_goal_no
de_id)
        else:
            route_tracking_task =
self.navigator.getAndTrackRoute(start=start_pose,goal=msg,use_start=False)
            task_canceled = False
            last_feedback = None
            follow_path_task = RunningTask.NONE
            while not self.navigator.isTaskComplete(task=route_tracking_task):
```

```

        feedback = self.navigators.getFeedback(task=route_tracking_task)
        while feedback is not None:
            if not last_feedback or (feedback.last_node_id !=
last_feedback.last_node_id or feedback.next_node_id !=
last_feedback.next_node_id):
                self.get_logger().info('Passed node ' +
str(feedback.last_node_id) +
' to next node ' + str(feedback.next_node_id) +
' along edge ' + str(feedback.current_edge_id) +
'.')

            last_feedback = feedback
            if feedback.rerouted: # or follow_path_task ==
RunningTask.None
                self.get_logger().info('Passing new route to
controller!')

                follow_path_task =
self.navigators.followPath(feedback.path)
                feedback =
self.navigators.getFeedback(task=route_tracking_task)
                if self.navigators.isTaskComplete(task=follow_path_task):
                    self.get_logger().info('Controller or waypoint follower
server completed its task!')
                    self.navigators.cancelTask()
                    task_canceled = True
                while not self.navigators.isTaskComplete(task=follow_path_task) and
not task_canceled:
                    time.sleep(0.1)

                result = self.navigators.getResult()
                if result == TaskResult.SUCCEEDED:
                    if self.__adjust_pose(msg):
                        self.get_logger().info(Fore.GREEN+"navigation task
completed."+Fore.RESET)

                        self.action_feedback_pub.publish(String(data="road_net_nav_succeeded"))
                    elif result == TaskResult.CANCELED:
                        self.get_logger().info(Fore.YELLOW+"navigation task
canceled."+Fore.RESET)
                    elif result == TaskResult.FAILED:
                        self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)

                        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))
                    else:
                        self.get_logger().info(Fore.RED+"navigation task
failed."+Fore.RESET)

                        self.action_feedback_pub.publish(String(data="road_net_nav_failed"))

            def __adjust_pose(self, final_pose)->bool:
                '''Adjust to target posture'''
                self.get_logger().info("adjusting final orientation...")
                final_orientation_task = self.navigators.goToPose(final_pose)
                while not self.navigators.isTaskComplete(task=final_orientation_task):
                    time.sleep(0.1)

                orientation_result = self.navigators.getResult()
                if orientation_result == TaskResult.SUCCEEDED:

```

```

        return True
    else:
        self.get_logger().info("final orientation adjustment failed.")
        return False

def __find_nearest_node(self, x, y):
    '''Use KDTree to find the nearest node'''
    distance, index = self.tree.query([x, y])
    nearest_node_id = self.node_ids[index]
    return nearest_node_id, distance

```

- The `__load_roadnet_nodes` method is used to parse **GeoJSON format** road network files during the program initialization stage, extract `Point` type nodes (topological nodes of the road network) from the road network, store node coordinates and IDs, and build `KDTree` index.

```

def __load_roadnet_nodes(self):
    '''Load road network nodes for KDTree search'''
    with open(self.road_net_file, 'r') as f:
        data = json.load(f)

    self.node_coords = []    # [[x, y], ...]
    self.node_ids = []      # Corresponding node ID

    for feat in data['features']:
        if feat['geometry']['type'] == 'Point':
            x, y = feat['geometry']['coordinates']
            self.node_coords.append([x, y])
            self.node_ids.append(feat['properties'].get("id", None))

    self.tree = KDTree(self.node_coords)
    self.get_logger().info(f"Loaded {len(self.node_coords)} nodes into KDTree")

```